SME Report: Quantum Data Loading & Dependencies

This report provides the specialized industry knowledge required to load biological discovery data into the **Origin Pilot OS** (QPanda) environment for "Layer 7" (Quantum) validation.

1. SME Dependency List (Industry Standard)

To interface your Python backend with the local Origin Pilot OS, you must ensure the following stack is initialized in your local environment:

Dependency	Purpose	Why it matches Origin Pilot OS
`pyqpanda`	Core Quantum Framework	Primary SDK for Origin Quantum's Pilot OS.
`pyqpanda-chem`	Quantum Chemistry	Required for VQE and Hamiltonian transformations.
`PySCF`	Integral Generator	Generates the molecular integrals needed for quantum Hamiltonians.
`RDKit`	Cheminformatics	Essential for converting your ChEMBL SMILES into XYZ 3D coordinates.
`Driver` **(Psi4/PySCF)**	Basis Set Mapping	Maps atoms to electronic orbitals (STO-3G, 6-31G).

2. Data Requirements for Quantum Loading

Loading data "into Quantum" isn't just about moving files; it's about preparing the **Hamiltonian**. You need to load:

A. Geometric Data (The "Input")

- **Ligand Coordinates (XYZ/SDF)**: Generated from backend/app/services/chembl_service.py SMILES.
- **Active Site Mask**: If using a "Quantum Embedding" approach, you need the coordinates of the protein residues within 5Å of the ligand.

B. Chemical Metadata

- **Charge & Multiplicity**: (e.g., Charge 0, Multiplicity 1 for most small molecules).
- **Basis Set Name**: (e.g., `sto-3g` for initial testing).

C. Validation Data

- **Exp. Binding Affinity**: From your `pchembl_value` fields, used to compare with Calculated Quantum Energy.

3. SME Advice for Local Loading

Since you are installing locally on a China-based model (Pilot OS):

1. **Use the `CPUQVM` first**: It mimics the quantum computer on your local CPU. Once verified, switch to the `GPUQVM` for speedup.
2. **Hamiltonian Compression**: For molecules > 10 atoms, use `JordanWigner` or `BravyiKitaev` transformations to fit the qubit memory limits of the OS.

Quantum Integration Plan (Origin Pilot OS / QPanda)

This plan outlines the steps to integrate local quantum simulation capabilities using the **Origin Pilot OS** (QPanda framework) into the existing Biologics Discovery Platform.

User Review Required

> [!IMPORTANT] Since you are using **Origin Pilot OS** (developed by Origin Quantum), we will utilize the **QPanda** framework instead of Qiskit for local compatibility.
>
> **Action Required**: Please confirm if you have the *pyqpanda* library installed in your local Python environment.

Proposed Changes

[Backend] Quantum Service Layer

[NEW] **quantum_service.py**

Create a new service to handle quantum molecular simulations.

- **Goal**: Implement a Variational Quantum Eigensolver (VQE) wrapper using *pyqpanda*.
- **Inputs**: Molecular structure (SMILES/XYZ), Basis set selection.
- **Output**: Refined ground state energy / Binding affinity score.

[Backend] Integration with Optimization

[MODIFY] **optimization.py**

- Integrate *quantum_service* to allow "Layer 7" validation for top-tier lead candidates.

Data SME: Required Data for Quantum Loading

To perform accurate quantum discovery, you must load the following data into the Quantum workspace:

3. **Structural Data**:
    - **SMILES**: For small molecule identity.
    - **PDB/SDF**: 3D coordinates of the ligand and protein active site (crucial for Hamiltonians).
4. **Quantum Chemistry Parameters**:
    - **Basis Sets**: (e.g., STO-3G for speed, 6-31G* for accuracy).
    - **Active Space**: Number of electrons and orbitals to include in the simulation.
5. **Hamiltonian Data**: Generated from the molecular geometry via *pyqpanda-chem* or similar utilities.

Local Installation Guide (Origin Pilot OS)

1. **Environment Setup**:

```
pip install pyqpanda
```

2. **Quantum Cluster Connection**:
   - If running on a local Origin Pilot OS cluster, ensure the *QCloud* or local *CPUQVM* / *GPUQVM* is initialized.
3. **Driver Dependencies**:
   - Ensure *psi4* or *PySCF* is installed if you need to generate molecular integrals for the quantum simulation.

Verification Plan

Automated Tests

- Run *pytest backend/tests/test_quantum_service.py* (to be created) using a *CPUQVM* simulator.

Manual Verification

- Verify that a sample SMILES (e.g., Caffeine/Water) can undergo a basic energy calculation without errors.